

Types of Recurrent Neural Networks (RNNs)

1 One-to-One RNN

This is the simplest form of RNN, where you have a single input and a single output. It does not exploit the sequential nature of the data, as there is no temporal relationship between inputs.

Mathematical Formulation: Let x_t represent the input at time step t , h_t be the hidden state, and y_t be the output at time step t .

The update rule for the hidden state is:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

The output is:

$$y_t = W_{hy}h_t + b_y$$

Code Implementation:

Listing 1: many-to-one

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleRNNOneToOne(nn.Module):
5     def __init__(self, input_size, hidden_size, output_size):
6         super(SimpleRNNOneToOne, self).__init__()
7         self.rnn = nn.RNN(input_size, hidden_size)
8         self.fc = nn.Linear(hidden_size, output_size)
9
10    def forward(self, x):
11        out, h = self.rnn(x)
12        out = self.fc(out)
13        return out
```

2 One-to-Many RNN

In a one-to-many RNN, you have a single input but multiple outputs. This is useful in tasks like generating a sequence (e.g., generating text or music) from an initial condition (e.g., a starting token).

Mathematical Formulation: The hidden state is updated as:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

The output sequence is generated for each time step t :

$$y_t = W_{hy}h_t + b_y$$

Code Implementation:

Listing 2: One-to-many

```

1 class SimpleRNNOneToMany(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(SimpleRNNOneToMany, self).__init__()
4         self.rnn = nn.RNN(input_size, hidden_size)
5         self.fc = nn.Linear(hidden_size, output_size)
6
7     def forward(self, x):
8         out, h = self.rnn(x)
9         out = self.fc(out) # Each time step has an output
10        return out

```

3 Many-to-One RNN

In a many-to-one RNN, you have a sequence of inputs and a single output. This architecture is used for tasks where you process an entire sequence and produce one output based on the sequence, such as sentiment analysis or classification.

Mathematical Formulation: For a sequence of inputs $\{x_1, x_2, \dots, x_T\}$, the hidden state is updated at each time step t :

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

At the end of the sequence, the output is:

$$y = W_{hy}h_T + b_y$$

Code Implementation:

Listing 3: many-to-one

```

1 class SimpleRNNManyToOne(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(SimpleRNNManyToOne, self).__init__()
4         self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
5         self.fc = nn.Linear(hidden_size, output_size)
6
7     def forward(self, x):
8         out, h = self.rnn(x)
9         out = self.fc(h[-1]) # Use the last hidden state for prediction
10        return out

```

4 Many-to-Many RNN

In this case, both the input and the output are sequences. This is used in tasks like sequence-to-sequence (seq2seq) models for machine translation or summarization, where the input and output sequences have the same length or different lengths.

Mathematical Formulation: For an input sequence $\{x_1, x_2, \dots, x_T\}$, the hidden state is updated at each time step t :

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

At each time step t , an output is produced:

$$y_t = W_{hy}h_t + b_y$$

Code Implementation:

Listing 4: many-to-many

```
1 class SimpleRNNManyToMany(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(SimpleRNNManyToMany, self).__init__()
4         self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
5         self.fc = nn.Linear(hidden_size, output_size)
6
7     def forward(self, x):
8         out, h = self.rnn(x)
9         out = self.fc(out) # Each time step produces an output
10        return out
```

5 Summary of RNN Architectures

- **One-to-One:** Single input and output, typically used in simple prediction tasks.
- **One-to-Many:** Single input with multiple outputs, used in generative models.
- **Many-to-One:** Multiple inputs with a single output, often used in classification tasks.
- **Many-to-Many:** Multiple inputs with multiple outputs, used in tasks like machine translation or summarization.

These RNN architectures differ in terms of how input and output sequences are structured and are typically chosen based on the specific task and data type.

RNN parameters

When using a Recurrent Neural Network (RNN) with an embedding layer at the beginning, the weight matrices of the RNN can be understood based on the transformations it performs at each time step. Lets break it down step by step.

Embedding Layer

Purpose

The embedding layer transforms discrete indices (e.g., word or character indices) into dense, continuous vectors of a fixed size, called the embedding size (`embed_size`).

- **Input shape:** (`batch_size`, `seq_length`) (indices for words or tokens).
- **Output shape:** (`batch_size`, `seq_length`, `embed_size`).

Weights

The embedding layer has a weight matrix of shape (`vocab_size`, `embed_size`). Each row corresponds to the embedding vector of a unique token in the vocabulary.

RNN Layer

Input to the RNN

After the embedding layer, the RNN receives input of shape (`batch_size`, `seq_length`, `embed_size`).

Core RNN Mechanics

At each time step t , the RNN processes one input vector x_t (of size `embed_size`) and updates the hidden state h_t (of size `hidden_size`).

The recurrence equation is:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

where:

- W_{ih} : Input-to-hidden weight matrix.
- W_{hh} : Hidden-to-hidden weight matrix.
- b_h : Bias term.
- h_t : Hidden state at time t .

Weights of the RNN

- **Input-to-Hidden Weights (W_{ih}):**
 - **Shape:** (`hidden_size`, `embed_size`).
 - Connects the input vector x_t to the hidden state.
- **Hidden-to-Hidden Weights (W_{hh}):**
 - **Shape:** (`hidden_size`, `hidden_size`).
 - Connects the previous hidden state h_{t-1} to the current hidden state h_t .
- **Bias (b_h):**
 - **Shape:** (`hidden_size`).
 - Bias added to the hidden state calculation.

Output

The RNN outputs a hidden state h_t at every time step t .

- If `batch_first=True`, the output shape is (`batch_size`, `seq_length`, `hidden_size`).